

Implémentation et Analyse de la Méthode SIFT (Scale-Invariant Feature Transform)

Louis LENOBLE, Raphael ROGER, Dylan SANSON, Jack SU

Projet TIM - Traitement d'Image

28 Novembre 2025

Résumé

Ce rapport présente une étude approfondie et une implémentation de l'algorithme SIFT (Scale-Invariant Feature Transform), initialement proposé par David G. Lowe en 2004. SIFT est une méthode robuste pour l'extraction de caractéristiques locales dans les images, invariante aux changements d'échelle, à la rotation et partiellement invariante aux changements d'illumination et au point de vue 3D. Nous détaillons les quatre étapes fondamentales de l'algorithme : la détection des extrêmes dans l'espace des échelles, la localisation précise des points clés, l'assignation d'orientation et la création des descripteurs. Nous présentons ensuite notre propre implémentation en Python, en analysant les choix techniques effectués. Enfin, nous discutons des applications possibles (telles que le stitching d'images et la réalité augmentée) ainsi que des limites intrinsèques de la méthode.

Table des matières

1	Introduction	3
1.1	Contexte et Problématique	3
1.2	Limites des Approches Historiques	3
1.3	La Proposition SIFT	4
1.4	Objectifs du Rapport	5
2	Théorie de la méthode SIFT	6
2.1	Détection des extremums dans l'espace des échelles	6
2.1.1	Espace des échelles et convolution Gaussienne	6
2.1.2	Différence de Gaussiennes (DoG)	6
2.1.3	Organisation en Octaves	7
2.2	Localisation précise des points clés	7
2.2.1	Raffinement Sub-pixel (Interpolation de Taylor)	8
2.2.2	Filtrage des points à faible contraste	8
2.2.3	Élimination des réponses de bords	9
2.3	Assignment d'orientation	9
2.4	Descripteur de point clé	10
2.4.1	Construction de la grille	10
2.4.2	Le vecteur final et sa normalisation	10
3	Analyse et Discussion des Résultats	12
3.1	Analyse de la Robustesse (Avantages)	12
3.1.1	Invariances Géométriques	12
3.1.2	Invariance Photométrique	12
3.1.3	Discrimination (Matching)	13
3.2	Limites Intrinsèques de la Méthode	13
3.2.1	Coût de Calcul et Complexité	13
3.2.2	Le problème des zones peu texturées	13
3.2.3	Invariance Affine Limitée	13
4	Implémentation Détaillée	14
4.1	1. Construction de l'Espace des Échelles (Scale-Space)	14
4.1.1	Génération des sigmas	14
4.1.2	Création des DoG	15
4.1.3	Détection des Extremums Locaux	15
4.2	2. Raffinement et Filtrage (Keypoint Localization)	15
4.2.1	Calcul des Gradients et de la Hessienne	15
4.2.2	Boucle de Raffinement (Newton-Raphson)	16
4.2.3	Filtrage des Arêtes (Edge Response)	16

4.3	3. Assignment d'Orientation	16
4.3.1	Calcul de l'histogramme	16
4.3.2	Lissage et Pics	17
4.4	Structure des données de sortie	17
5	Conclusion	18
A	Références	19

Chapitre 1

Introduction

La vision par ordinateur a pour but ultime de permettre aux machines de comprendre et d'interpréter le monde visuel. Au cœur de nombreux systèmes de vision moderne — qu'il s'agisse de la reconnaissance d'objets, de la reconstruction 3D, de la création de panoramas (stitching) ou de la navigation robotique — se trouve un problème fondamental : la mise en correspondance d'images (*image matching*).

Pour identifier qu'un objet présent dans une image A est le même que celui dans une image B, l'algorithme doit être capable d'identifier des points d'intérêt communs, malgré les transformations drastiques que l'image a pu subir entre les deux prises de vue.

1.1 Contexte et Problématique

Les images du monde réel sont soumises à de nombreuses variations qui rendent la comparaison pixel par pixel impossible. Un système de reconnaissance robuste doit être capable de gérer :

- **Les changements d'échelle** : L'objet peut être proche ou lointain.
- **Les rotations** : L'objet ou la caméra peuvent avoir tourné.
- **Les changements d'illumination** : La lumière peut varier en intensité ou en direction.
- **Le point de vue 3D** : Une perspective différente déforme l'apparence 2D de l'objet (distorsion affine).
- **Le bruit et les occultations** : L'image peut être dégradée ou l'objet partiellement caché.

L'objectif est donc d'extraire des "caractéristiques" (*features*) locales qui sont invariantes à ces transformations. Ces caractéristiques doivent être suffisamment distinctives pour ne pas être confondues avec d'autres points de l'image.

1.2 Limites des Approches Historiques

Avant l'avènement de SIFT, les méthodes de détection de points d'intérêt se basaient principalement sur la détection de coins (*corners*). L'algorithme le plus célèbre, le détecteur de Harris-Stephens (1988), identifie les régions où l'intensité de l'image change significativement dans toutes les directions.

Bien que le détecteur de Harris soit très efficace pour l'invariance en rotation (un coin reste un coin même si l'image tourne), il souffre d'une limitation majeure : il n'est pas

invariant à l'échelle.

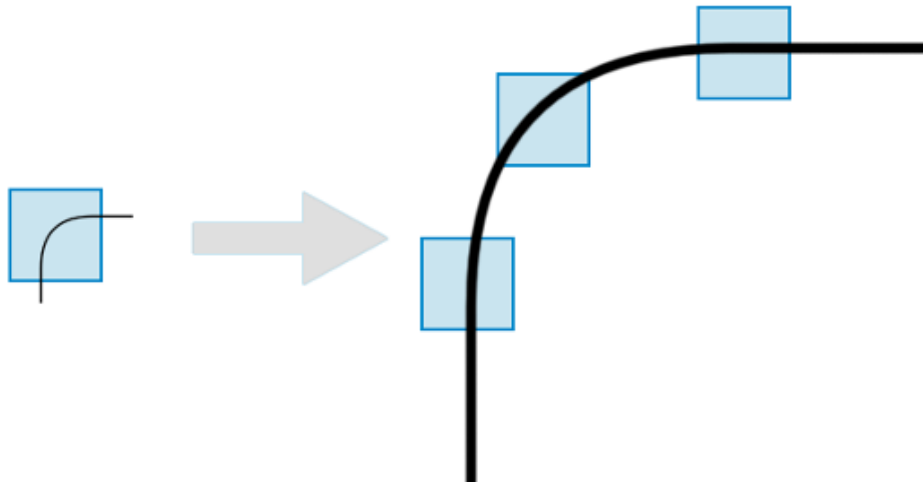


FIGURE 1.1 – *Illustration du problème d'invariance d'échelle avec le détecteur de Harris. Les coins détectés à une échelle ne sont pas retrouvés à une autre échelle. Extrait de Lowe, D. G. (2004). Distinctive Image Features from Scale-Invariant Keypoints. International Journal of Computer Vision.*

Comme nous l'avons souligné dans notre présentation, pour repérer des coins à différentes échelles avec Harris, il faudrait adapter manuellement la taille de la fenêtre de détection, ce qui est une tâche fastidieuse et peu robuste pour une automatisation complète.

1.3 La Proposition SIFT

C'est dans ce contexte que David G. Lowe a proposé, dans son papier fondateur de 2004 "*Distinctive Image Features from Scale-Invariant Keypoints*", la méthode SIFT (Scale-Invariant Feature Transform).

L'idée révolutionnaire de SIFT est de ne pas analyser l'image à une seule résolution, mais de construire un **espace des échelles** (*scale-space*). En recherchant des extremums stables à travers toutes les échelles possibles, SIFT permet d'identifier des points d'intérêt qui sont intrinsèquement liés à la taille des structures de l'image.

Selon Lowe, les caractéristiques SIFT possèdent des propriétés remarquables :

1. **Invariance** : Elles sont invariantes à l'échelle et à la rotation, et font preuve d'une grande robustesse face aux changements d'illumination et au bruit.
2. **Distinctivité** : Chaque point clé est associé à un descripteur riche (vecteur de 128 dimensions) qui permet de le différencier avec une très haute probabilité parmi une grande base de données de features.
3. **Quantité** : La méthode génère un grand nombre de features (même sur de petits objets), ce qui augmente la fiabilité de la reconnaissance, même en cas d'occultation partielle.

1.4 Objectifs du Rapport

Ce rapport a pour but de décortiquer le fonctionnement de cet algorithme emblématique. Notre démarche s'articule autour de trois axes :

1. **Compréhension Théorique** : Analyser les fondements mathématiques décrits par Lowe (Différence de Gaussiennes, développement de Taylor, histogrammes de gradients).
2. **Implémentation Pratique** : Recréer l'algorithme SIFT "from scratch" en Python. L'objectif n'est pas d'utiliser la fonction toute faite 'cv2.SIFT' d'OpenCV, mais de réécrire la logique de construction des octaves, de localisation des extrema et de création des descripteurs pour maîtriser les concepts sous-jacents.
3. **Expérimentation** : Tester notre implémentation sur des images standards (comme "Box" ou "Cameraman"), visualiser les résultats (orientation, échelle) et discuter des limites rencontrées lors du développement.

Chapitre 2

Théorie de la méthode SIFT

L'approche de SIFT décrite par David G. Lowe se décompose en quatre étapes séquentielles :

1. **Détection des extremums dans l'espace des échelles** : Recherche initiale des points d'intérêt sur toutes les échelles et positions possibles.
2. **Localisation des points clés** : Raffinement précis de la position et suppression des points instables (faible contraste ou situés sur des arêtes).
3. **Assignment d'orientation** : Attribution d'une ou plusieurs orientations basées sur le gradient local pour assurer l'invariance par rotation.
4. **Descripteur de point clé** : Création d'une signature vectorielle unique représentant le voisinage du point.

2.1 Détection des extremums dans l'espace des échelles

La première difficulté de la vision par ordinateur est de repérer le même objet à différentes distances. Un "coin" net à une certaine distance peut devenir une tache floue s'il est vu de loin. Pour résoudre ce problème, SIFT recherche des structures stables à travers un *espace des échelles* continu.

2.1.1 Espace des échelles et convolution Gaussienne

L'espace des échelles d'une image est une fonction $L(x, y, \sigma)$ obtenue par la convolution de l'image originale $I(x, y)$ avec un noyau Gaussien $G(x, y, \sigma)$ de largeur variable σ :

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y) \quad (2.1)$$

où $G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$. Plus σ est grand, plus l'image est floutée, simulant une observation de l'objet à une plus grande distance (perte des détails haute fréquence).

2.1.2 Différence de Gaussiennes (DoG)

Pour détecter des points d'intérêt caractéristiques (comme des taches ou des coins), on cherche traditionnellement les extremums du Laplacien de Gaussienne ($\sigma^2 \nabla^2 G$). Cependant, cette opération est coûteuse en calcul. Lowe propose d'approximer ce Laplacien par une **Différence de Gaussiennes** (DoG), notée $D(x, y, \sigma)$, qui est beaucoup plus rapide à calculer :

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma) \quad (2.2)$$

2.1.3 Organisation en Octaves

Pour couvrir efficacement l'ensemble des échelles possibles, l'espace des échelles est discretisé en **octaves**.

- Une octave correspond à un doublement de la valeur de σ .
- Chaque octave est divisée en s intervalles (ou sous-niveaux) entiers.

Pour détecter des extremums dans s intervalles, nous avons besoin de calculer les différences DoG. Pour obtenir s images DoG, nous devons générer $s + 1$ images DoG, et donc disposer de $s + 3$ images gaussiennes (L) par octave. Une fois une octave traitée, l'image gaussienne située à 2σ est sous-échantillonnée par un facteur 2 (un pixel sur deux est gardé) pour servir d'image de base à l'octave suivante. Cela réduit considérablement la complexité algorithmique.

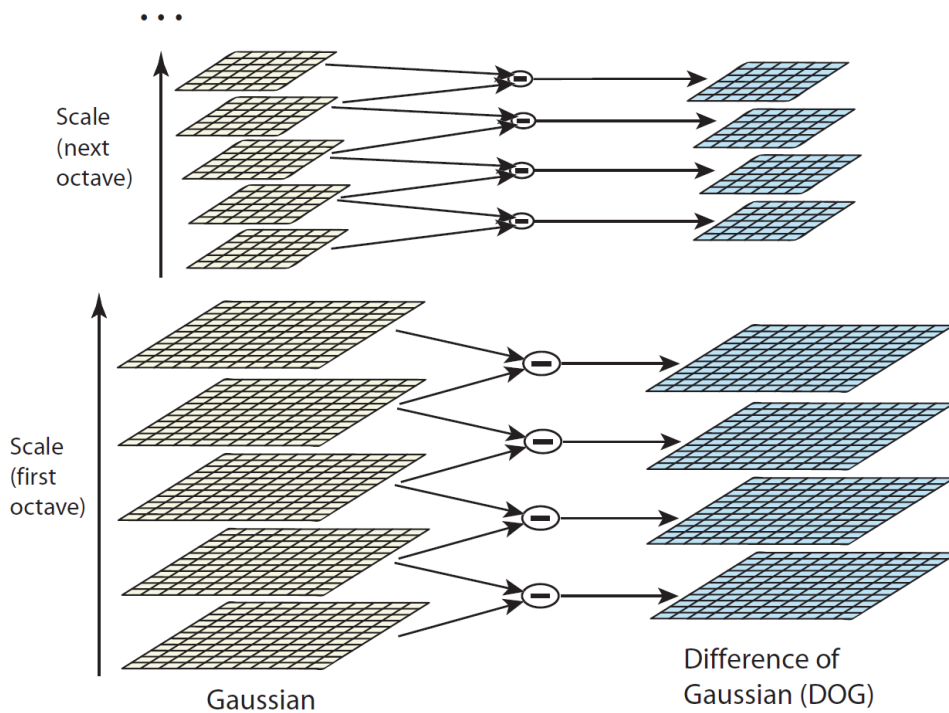


FIGURE 2.1 – Schéma de la pyramide gaussienne et des différences de gaussiennes (DoG). Extrait de la documentation OpenCV sur l'implémentation SIFT.

2.2 Localisation précise des points clés

Une fois les images DoG générées, l'algorithme cherche les extremums locaux. Un pixel est considéré comme un candidat s'il est strictement supérieur (ou strictement inférieur) à ses 8 voisins dans l'image courante, ainsi qu'à ses 9 voisins dans l'échelle supérieure et ses 9 voisins dans l'échelle inférieure (soit 26 voisins au total).

Cependant, cette détection brute produit beaucoup de points erronés ou mal localisés.

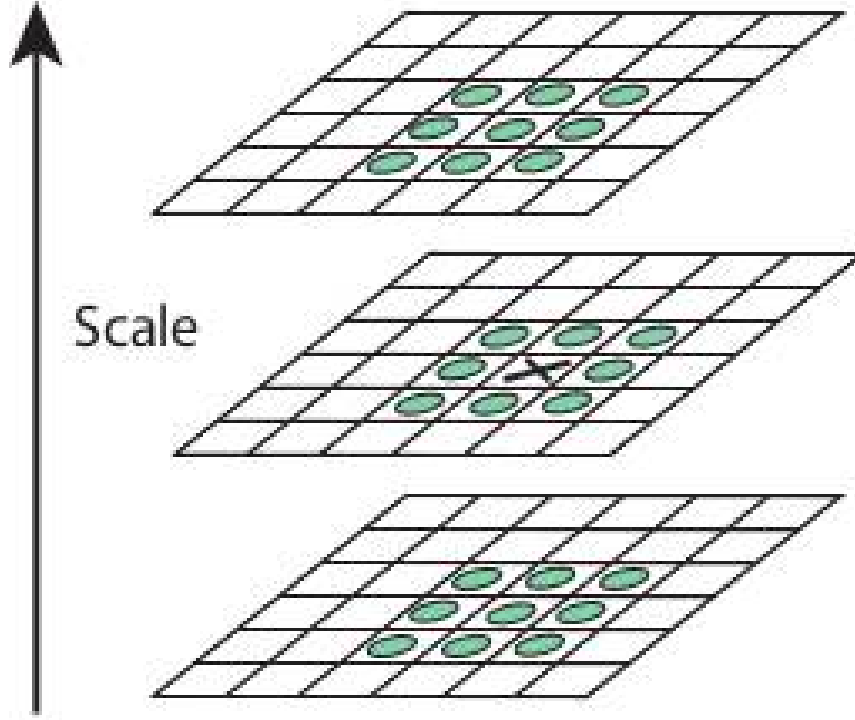


FIGURE 2.2 – *Illustration de la détection des extremums locaux dans l'espace des échelles. Extrait de la documentation OpenCV sur l'implémentation SIFT.*

2.2.1 Raffinement Sub-pixel (Interpolation de Taylor)

Les extremums détectés sont initialement positionnés sur la grille discrète des pixels. Pour une précision sub-pixel (nécessaire pour un matching précis), SIFT modélise la fonction DoG localement par un développement de Taylor du second ordre :

$$D(\mathbf{x}) = D + \frac{\partial D}{\partial \mathbf{x}}^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \frac{\partial^2 D}{\partial \mathbf{x}^2} \mathbf{x} \quad (2.3)$$

où $\mathbf{x} = (x, y, \sigma)^T$ est le décalage par rapport au point d'échantillonnage. La position exacte de l'extremum $\hat{\mathbf{x}}$ est trouvée en annulant la dérivée de cette fonction par rapport à $\mathbf{x}$:

$$\hat{\mathbf{x}} = - \left(\frac{\partial^2 D}{\partial \mathbf{x}^2} \right)^{-1} \frac{\partial D}{\partial \mathbf{x}} \quad (2.4)$$

Si le décalage calculé $\hat{\mathbf{x}}$ est supérieur à 0.5 dans une direction, le point d'interpolation se trouve plus près d'un voisin : l'algorithme déplace alors le point central et réitère le calcul.

2.2.2 Filtrage des points à faible contraste

Les points situés dans des zones homogènes sont sensibles au bruit. On évalue la valeur de la fonction DoG au point interpolé $D(\hat{\mathbf{x}})$. Si cette valeur est trop faible (typiquement $|D(\hat{\mathbf{x}})| < 0.03$ pour des pixels normalisés entre 0 et 1), le point est rejeté.

2.2.3 Élimination des réponses de bords

C'est une étape cruciale. La fonction DoG réagit fortement sur les arêtes (bords), tout comme un détecteur de contours. Or, un point situé sur une ligne droite est mal localisé dans la direction de la ligne (problème d'ouverture). On veut conserver uniquement les "coins" ou les "taches".

Pour cela, on analyse la courbure principale via la matrice Hessienne 2×2 , H , calculée au point clé :

$$H = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix} \quad (2.5)$$

Les valeurs propres de H sont proportionnelles aux courbures principales. Plutôt que de calculer explicitement ces valeurs propres, SIFT utilise une astuce algébrique basée sur la trace ($Tr(H) = D_{xx} + D_{yy}$) et le déterminant ($Det(H) = D_{xx}D_{yy} - (D_{xy})^2$). Pour rejeter les arêtes, on vérifie si le rapport des courbures principales est inférieur à un seuil r (Lowe utilise $r = 10$) via la condition suivante :

$$\frac{Tr(H)^2}{Det(H)} < \frac{(r+1)^2}{r} \quad (2.6)$$

Si cette inégalité n'est pas vérifiée, cela signifie que la courbure est très forte dans une direction et faible dans l'autre (cas typique d'une arête) : le point est alors supprimé.

2.3 Assignment d'orientation

À ce stade, nous avons des points stables ($\mathbf{x}$) à une échelle précise (σ). Pour rendre le descripteur invariant à la rotation de l'image, on attribue une orientation canonique à chaque point clé.

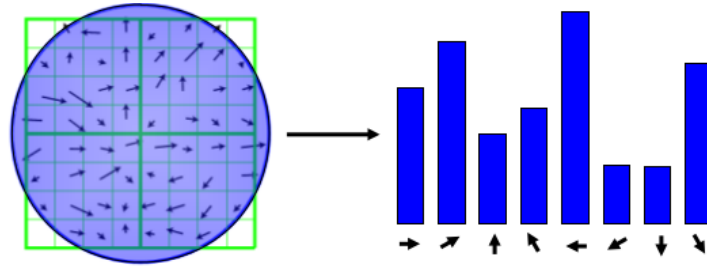


FIGURE 2.3 – Calcul de l'orientation principale autour du point clé. Extrait de l'étude de Liu et al. (2008).

Les calculs sont effectués sur l'image gaussienne L la plus proche de l'échelle du point clé, afin que les calculs de gradients soient cohérents avec le niveau de flou de la structure détectée. Pour chaque pixel (x, y) dans un voisinage autour du point clé, on calcule la magnitude $m(x, y)$ et l'orientation $\theta(x, y)$ du gradient :

$$m(x, y) = \sqrt{(L_{x+1,y} - L_{x-1,y})^2 + (L_{x,y+1} - L_{x,y-1})^2} \quad (2.7)$$

$$\theta(x, y) = \tan^{-1} \left(\frac{L_{x,y+1} - L_{x,y-1}}{L_{x+1,y} - L_{x-1,y}} \right) \quad (2.8)$$

Un histogramme d'orientation à 36 bins (couvrant les 360 degrés) est construit. Chaque échantillon ajouté à l'histogramme est pondéré par :

1. Sa magnitude $m(x, y)$.
2. Une fenêtre gaussienne circulaire centrée sur le point clé (avec $\sigma_{win} = 1.5 \times \sigma_{keypoint}$), pour donner plus d'importance aux gradients proches du centre.

Le pic le plus haut de l'histogramme définit l'orientation principale. Si des pics secondaires dépassent 80% de la hauteur du pic principal, un nouveau point clé est créé au même endroit et à la même échelle, mais avec cette orientation secondaire. Cela permet à SIFT de gérer de manière robuste les coins ambigus.

2.4 Descripteur de point clé

La dernière étape consiste à calculer une empreinte numérique (le descripteur) qui soit très distinctive tout en restant robuste aux variations locales (petits changements de perspective, illumination).

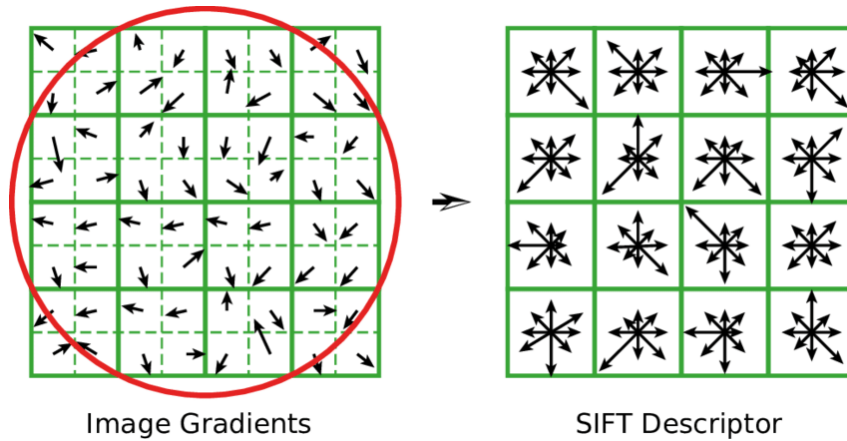


FIGURE 2.4 – Construction du descripteur SIFT à partir des histogrammes de gradients locaux. Extrait de la documentation OpenCV sur l'implémentation SIFT.

2.4.1 Construction de la grille

On considère une région carrée autour du point clé. **Point crucial** : cette région est orientée selon l'angle principal calculé à l'étape précédente. C'est cette rotation du système de coordonnées qui garantit l'invariance par rotation.

La région est divisée en une grille de 4×4 sous-régions. Dans chaque sous-région, on compile un histogramme d'orientation sur 8 directions (points cardinaux et intercardinaux). Afin d'éviter les effets de frontières brusques (quand un gradient passe d'une sous-région à l'autre), la contribution de chaque pixel est distribuée par interpolation trilinéaire (sur x , y et l'angle θ).

2.4.2 Le vecteur final et sa normalisation

Le descripteur final est la concaténation de ces histogrammes :

$$4 \times 4 \text{ sous-régions} \times 8 \text{ orientations} = 128 \text{ dimensions}$$

Ce vecteur v subit enfin une procédure de normalisation pour l'invariance photométrique :

1. **Normalisation unitaire** : $v \leftarrow v / \|v\|$. Cela compense les changements linéaires de contraste (gain global).
2. **Seuillage** : Les valeurs du vecteur supérieures à 0.2 sont plafonnées à 0.2. Cela réduit l'influence des effets d'illumination non-linéaires (comme les reflets spéculaires ou la saturation de la caméra).
3. **Renormalisation** : Le vecteur est à nouveau normalisé à une longueur unitaire.

Le résultat est un vecteur de 128 flottants, prêt à être comparé par distance Euclidienne pour le matching.

Chapitre 3

Analyse et Discussion des Résultats

Au-delà de l'implémentation technique, il est essentiel d'analyser pourquoi SIFT est devenu un standard industriel pendant deux décennies, mais aussi de comprendre les contraintes qui ont mené au développement de méthodes alternatives (comme SURF ou ORB).

3.1 Analyse de la Robustesse (Avantages)

L'avantage majeur de SIFT réside dans la "distinctivité" de ses descripteurs. Là où des méthodes plus anciennes se contentaient de corrélérer des zones de pixels (sensibles aux changements d'intensité), SIFT transforme l'information locale en une signature vectorielle de haute dimension.

3.1.1 Invariances Géométriques

- **Invariance d'Échelle et de Rotation** : C'est la promesse centrale de l'algorithme. Grâce à l'espace des échelles (pyramide gaussienne) et à l'assignation d'une orientation canonique, un objet peut être reconnu même s'il apparaît 5 fois plus petit ou s'il a subi une rotation quelconque.
- **Robustesse aux déformations affines** : Bien que SIFT ne modélise pas explicitement les transformations affines complètes (étirement, cisaillement), Lowe a démontré expérimentalement que les descripteurs restent robustes pour des changements de point de vue allant jusqu'à 50 degrés. Au-delà, la déformation des motifs locaux devient trop importante pour que les histogrammes de gradients restent corrélés.

3.1.2 Invariance Photométrique

Notre étude du descripteur a montré que SIFT est particulièrement résistant aux variations d'éclairage, grâce à une stratégie de normalisation en deux temps :

1. **Changement linéaire (Contraste/Luminosité)** : La normalisation du vecteur descripteur à une longueur unitaire ($\|v\| = 1$) annule les changements multiplicatifs d'intensité.
2. **Saturation non-linéaire** : En plafonnant les valeurs du vecteur à 0.2, SIFT réduit l'impact des effets non-linéaires importants, comme les reflets spéculaires ou la saturation du capteur de la caméra.

3.1.3 Discrimination (Matching)

La dimensionnalité élevée (128) du vecteur permet une discrimination très fine. Lowe suggère d'utiliser le rapport de distance (L_2) entre le plus proche voisin et le second plus proche voisin (NNDR - *Nearest Neighbor Distance Ratio*). Si ce ratio est inférieur à 0.8, le match est considéré comme valide. Cette méthode permet d'éliminer plus de 90% des faux positifs tout en conservant plus de 95% des correspondances correctes.

3.2 Limites Intrinsèques de la Méthode

Malgré ses performances, notre implémentation et la littérature mettent en évidence plusieurs goulots d'étranglement.

3.2.1 Coût de Calcul et Complexité

SIFT est un algorithme lourd.

- **Construction de l'espace des échelles** : La nécessité de réaliser de nombreuses convolutions gaussiennes et soustractions sur plusieurs octaves est coûteuse en temps CPU.
- **Dimension du descripteur** : Le vecteur de 128 flottants est lourd à stocker et à comparer. Dans une base de données contenant des milliers d'images, la recherche des plus proches voisins (même avec des structures optimisées comme les arbres k-d) reste une opération critique. C'est ce qui a motivé la création de descripteurs binaires comme BRIEF ou ORB, beaucoup plus rapides à comparer via la distance de Hamming.

3.2.2 Le problème des zones peu texturées

Comme nous l'avons observé sur l'image "Cameraman", SIFT dépend de la présence de gradients locaux.

- **Zones uniformes** : Le ciel ou les surfaces lisses ne génèrent pas d'extremums dans la différence de Gaussiennes.
- **Problème d'ouverture** : Bien que le rejet des arêtes (via la Hessienne) soit efficace, il signifie que SIFT ne peut pas exploiter les contours rectilignes (comme les bords d'un bâtiment) pour le matching, se limitant aux coins et aux taches texturées.

3.2.3 Invariance Affine Limitée

SIFT approxime les régions d'intérêt par des cercles. Lorsque la caméra change d'angle de vue de manière drastique (vue rasante), ces cercles devraient idéalement devenir des ellipses. SIFT ne compensant pas cette déformation elliptique, la performance chute drastiquement au-delà de 50 à 60 degrés d'angle. Des extensions comme *ASIFT* (*Affine-SIFT*) résolvent ce problème en simulant toutes les vues affines possibles, mais au prix d'une explosion du temps de calcul.

Chapitre 4

Implémentation Détaillée

Dans ce chapitre, vous retrouverez la façon dont nous avons implémenté les 3 premières étapes de SIFT (à la demande de notre professeur, Monsieur R. Condat). Nous avons fait le choix de développer l'algorithme en Python en limitant l'usage de bibliothèques externes.

- **Numpy** : Utilisé pour toutes les opérations matricielles (algèbre linéaire, calculs de gradients, histogrammes).
- **OpenCV (cv2)** : Utilisé strictement pour les opérations de bas niveau (chargement d'image, redimensionnement, et flou gaussien basique), afin de ne pas réinventer la convolution.
- **Matplotlib** : Pour la visualisation des résultats intermédiaires.

4.1 1. Construction de l'Espace des Échelles (Scale-Space)

La première étape consiste à générer la pyramide d'images. Dans notre code, cela est géré par la fonction `build_gaussian_octaves`.

4.1.1 Génération des sigmas

Pour chaque octave, nous devons générer une série d'images floutées. Comme spécifié dans la théorie, le facteur multiplicatif entre deux échelles est $k = 2^{1/s}$. La fonction `build_gaussian_kernels` pré-calcule les valeurs de σ nécessaires.

```
1 def build_gaussian_kernels(sigma, num_scales):
2     k = 2 ** (1.0 / num_scales)
3     gaussian_kernels = np.zeros(num_scales + 3)
4     gaussian_kernels[0] = sigma
5
6     for i in range(1, num_scales + 3):
7         # Calcul du sigma du noyau      appliquer pour passer de l'image
8         # précédente la suivante
9         sigma_prev = (k ** (i - 1)) * sigma
10        sigma_total = k * sigma_prev
11        gaussian_kernels[i] = np.sqrt(sigma_total ** 2 - sigma_prev **
12        2)
13
14    return gaussian_kernels
```

Listing 4.1 – Calcul des sigmas pour une octave

4.1.2 Création des DoG

La fonction `generateDoGImages` prend une octave (une liste d'images gaussiennes) et calcule les différences pixel à pixel entre images adjacentes. C'est ici que l'approximation du Laplacien est réalisée.

```
1 def generateDoGImages(gaussian_images):
2     dog_images = []
3     for i in range(len(gaussian_images) - 1):
4         # Soustraction simple des matrices numpy
5         dog_images.append(gaussian_images[i+1] - gaussian_images[i])
6     return dog_images
```

Listing 4.2 – Calcul des Différences de Gaussiennes

4.1.3 Détection des Extremums Locaux

L'étape de recherche brute est implémentée dans la fonction `detect_keypoints_in_dog`. Nous itérons sur chaque octave, puis sur chaque échelle s (de 1 à $s-1$), et enfin sur chaque pixel (i, j) de l'image (en excluant les bords pour éviter les erreurs d'index).

La fonction critique est `is_extremum`, qui vérifie si un pixel est strictement supérieur ou strictement inférieur à ses 26 voisins :

- 8 voisins dans l'échelle courante (s).
- 9 voisins dans l'échelle inférieure ($s-1$).
- 9 voisins dans l'échelle supérieure ($s+1$).

```
1 def is_extremum(first_subimage, second_subimage, third_subimage,
2                 threshold):
3     center_pixel_val = second_subimage[1, 1]
4
5     # Seuil préliminaire pour gagner du temps (si le point est trop
6     # faible, on l'ignore)
7     if abs(center_pixel_val) < threshold:
8         return False
9
10    # Comparaison avec les 26 voisins
11    # (Combinaison des 3 sous-blocs 3x3)
12    if center_pixel_val > 0:
13        return np.all(center_pixel_val >= first_subimage) and \
14               np.all(center_pixel_val >= third_subimage) and \
15               np.all(center_pixel_val >= second_subimage)
16    # ... (idem pour le cas < 0)
```

Listing 4.3 – Vérification des 26 voisins

4.2 2. Raffinement et Filtrage (Keypoint Localization)

C'est la partie la plus mathématique de l'implémentation. Nous avons codé les dérivées partielles à l'aide de différences finies centrales.

4.2.1 Calcul des Gradients et de la Hessienne

Pour localiser le point avec une précision sub-pixel, nous avons besoin du gradient $\frac{\partial D}{\partial \mathbf{x}}$ et de la Hessienne $\frac{\partial^2 D}{\partial \mathbf{x}^2}$. Nous avons implémenté ces calculs manuellement :

```

1 def compute_gradient_at_center(pixel_array):
2     # dx = (f(x+1) - f(x-1)) / 2
3     dx = 0.5 * (pixel_array[1, 1, 2] - pixel_array[1, 1, 0])
4     dy = 0.5 * (pixel_array[1, 2, 1] - pixel_array[1, 0, 1])
5     ds = 0.5 * (pixel_array[2, 1, 1] - pixel_array[0, 1, 1])
6     return np.array([dx, dy, ds])
7
8 def compute_hessian_at_center(pixel_array):
9     center_val = pixel_array[1, 1, 1]
10    # dxx = f(x+1) - 2f(x) + f(x-1)
11    dxx = pixel_array[1, 1, 2] - 2 * center_val + pixel_array[1, 1, 0]
12    dyy = pixel_array[1, 2, 1] - 2 * center_val + pixel_array[1, 0, 1]
13    dss = pixel_array[2, 1, 1] - 2 * center_val + pixel_array[0, 1, 1]
14    # ... (calcul des d riv es crois es dxy, dxs, dys)
15    return np.array([[dxx, dxy, dxs], [dxy, dyy, dys], [dxs, dys, dss]])

```

Listing 4.4 – Approximation des dérivées par différences finies

4.2.2 Boucle de Raffinement (Newton-Raphson)

La fonction `localize_keypoint` tente de converger vers la position réelle de l'extremum. Elle résout itérativement le système $\hat{\mathbf{x}} = -H^{-1}\nabla D$. Si le décalage calculé est supérieur à 0.5 pixel dans une direction, le centre de la fenêtre est déplacé. Si l'algorithme ne converge pas après 5 itérations, le point est rejeté.

4.2.3 Filtrage des Arêtes (Edge Response)

Une fois le point localisé, nous appliquons le critère de la Hessienne 2x2 (seulement spatiale x, y) pour éliminer les arêtes.

```

1 # Hessienne xy seulement
2 H_xy = hessian[:2, :2]
3 trace_H = np.trace(H_xy)
4 det_H = np.linalg.det(H_xy)
5
6 # Condition de ratio (r=10 par d faut)
7 if det_H <= 0 or (trace_H ** 2) / det_H >= ((r + 1) ** 2) / r:
8     return None # Point rejet (c'est une ar te)

```

Listing 4.5 – Rejet des arêtes via Trace et Déterminant

4.3 3. Assignment d'Orientation

Pour chaque point clé validé, nous calculons son orientation dominante.

4.3.1 Calcul de l'histogramme

La fonction `compute_orientation_histogram` explore une fenêtre autour du point clé. La taille de cette fenêtre dépend de l'échelle du point clé ($radius = 3 \times 1.5 \times \sigma$). Pour chaque pixel de cette fenêtre, nous calculons la magnitude et l'orientation du gradient :

```

1 # Calcul vectoriel numpy
2 dx = L[y, x+1] - L[y, x-1]
3 dy = L[y+1, x] - L[y-1, x]

```

```

4 magnitude = np.sqrt(dx**2 + dy**2)
5 orientation = np.rad2deg(np.arctan2(dy, dx)) % 360

```

Listing 4.6 – Calcul Magnitude et Orientation

Ces valeurs remplissent un histogramme de 36 bins (10 degrés par bin), pondéré par une gaussienne centrée sur le point clé.

4.3.2 Lissage et Pics

L'histogramme brut est lissé (fonction `smooth_histogram`) pour réduire le bruit. Enfin, la fonction `assign_orientations` identifie le pic maximal. Si des pics secondaires dépassent 80% du pic maximal, nous dupliquons le point clé avec cette nouvelle orientation, ce qui améliore la robustesse de la détection.

4.4 Structure des données de sortie

À la fin du pipeline, nos points clés sont stockés sous forme d'objets `cv2.KeyPoint`. Cela nous permet d'utiliser la fonction `cv2.drawKeypoints` pour visualiser facilement nos résultats et comparer notre implémentation "maison" avec celle d'OpenCV.

```

1 new_kp = cv2.KeyPoint()
2 new_kp.pt = tuple(position_reelle * (2 ** octave_index)) # Remise l'
   chelle originale
3 new_kp.size = sigma * (2 ** (octave_index + 1))
4 new_kp.angle = peak_angle
5 new_kp.response = abs(extremum_val)
6 keypoints.append(new_kp)

```

Listing 4.7 – Création de l'objet KeyPoint final

Chapitre 5

Conclusion

Le projet nous a permis de décortiquer l'algorithme SIFT. En passant de la théorie mathématique (espace des échelles, Hessienne) à l'implémentation pratique en Python, nous avons pu constater la finesse de la méthode proposée par Lowe. La combinaison d'une détection multi-échelle et d'une description locale robuste fait de SIFT une pierre angulaire de la vision par ordinateur moderne, malgré l'émergence récente de méthodes basées sur le Deep Learning.

Annexe A

Références

1. Lowe, D. G. (2004). Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision*.
2. Support de présentation "Traitement d'image : SIFT", Projet TIM, Louis LE-NOBLE, Raphael ROGER, Dylan SANSON, Jack SU, 2025.
3. Documentation OpenCV sur l'implémentation SIFT : https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html
4. Liu, C., Yuen, J., Torralba, A. (2008). SIFT Flow : Dense Correspondence across Different Scenes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
5. Wikipedia - Scale-invariant feature transform : https://fr.wikipedia.org/wiki/Scale-invariant_feature_transform